BACKEND INTERNSHIP — TASK BRIEF

Database & Mongoose CRUD

Swap out in-memory data for a real MongoDB database and build persistent CRUD operations with Mongoose.

DURATION	STACK	DIFFICULTY
7 Days	**MongoDB, Mongoose**	**Beginner – Intermediate**

Goal of the Week

This week, Slotify gets a real database. You'll connect your Express app to MongoDB using Mongoose, replace all in-memory data with persistent storage, and rebuild your slot routes on top of proper database operations. By the end of the week, slots created through the API will actually survive a server restart — which is what "real" means in backend development.

Tasks

1. **MongoDB setup.** Create a free MongoDB Atlas cluster (or run MongoDB locally). Get your connection string and store it safely in a `.env` file — never commit this to GitHub.
2. **Install dependencies.** Add `mongoose` and `dotenv` to the project. Connect to MongoDB on server startup and log a success or error message.
3. **Slot schema.** Define a Mongoose schema for a slot with fields: `date`, `time`, `duration`, `isBooked`, and `createdAt`. Export it as a model.
4. **Rebuild GET /slots.** Replace the in-memory array with `Slot.find()` to return all slots from the database.
5. **Rebuild GET /slots/:id.** Use `Slot.findById()` and return a proper 404 if the slot doesn't exist.
6. **Rebuild POST /slots.** Validate the request body and use `Slot.create()` to save a new slot. Return 201 on success.
7. **Rebuild DELETE /slots/:id.** Use `Slot.findByIdAndDelete()` and handle the case where the ID doesn't exist.
8. **Add PUT /slots/:id.** Build an update route using `Slot.findByIdAndUpdate()` to edit slot details.

Suggested Daily Pace

DAY 1–2

Set up MongoDB Atlas, install mongoose + dotenv, connect to DB on startup.

DAY 3–4

Define Slot schema and model, rebuild GET routes with real DB queries.

DAY 5–6

Rebuild POST, DELETE, and add new PUT /slots/:id route.

DAY 7

Test all routes end-to-end, confirm data persists on restart, commit + push.

Resources

MongoDB Atlas	mongodb.com/cloud/atlas — free tier is more than enough for this project
Mongoose Docs	mongoosejs.com/docs/guide.html — Schema, Model, and CRUD methods
dotenv	npmjs.com/package/dotenv — keeping secrets out of your codebase
API Testing	Postman or Thunder Client — test all 5 routes with real DB data

Submission Checklist

- [ ] MongoDB connection established and logged on server startup
- [ ] Slot schema defined with all required fields
- [ ] All 5 routes (GET all, GET one, POST, PUT, DELETE) work with real DB data
- [ ] Data persists correctly after server restart
- [ ] Connection string stored in .env and .env is in .gitignore
- [ ] Code pushed to GitHub with at least 3 meaningful commits, repo link shared by end of Day 7